

AI-Based Predictive Maintenance for CNC Milling Machines

A software-only predictive maintenance system using Random Forest, with explainable predictions and a SCADA operator interface

The system predicts which of four failure mechanisms a machine is heading toward a mean of **8.67 operating hours** before the failure occurs, warning in advance on **97.6%** of unseen machines.

Submitted by

USN: _____

USN: _____

USN: _____

USN: _____

Under the guidance of

Department

Electronics & Communication Engineering

Institution

Government Engineering College, K. R. Pete

Academic year

2026 - 2027

Government Engineering College, K. R. Pete
Department of Electronics & Communication Engineering

CERTIFICATE

This is to certify that the project work entitled “**AI-Based Predictive Maintenance for CNC Milling Machines**” is a bonafide work carried out by the students named below, in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Electronics and Communication Engineering, of the Visvesvaraya Technological University, Belagavi, during the academic year 2026-2027.

It is certified that all corrections and suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Signature of Guide	Signature of HOD	Signature of Principal
Name & Designation	Name & Designation	Name & Designation

External Viva

Name of the Examiners	Signature with Date
1.	
2.	

Declaration

We hereby declare that the project work entitled “**AI-Based Predictive Maintenance for CNC Milling Machines**” has been carried out by us and submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Electronics and Communication Engineering.

We further declare that the dataset used is the publicly available **AI4I 2020 Predictive Maintenance Dataset** from the UCI Machine Learning Repository, that all software was written by us, and that all sources consulted have been acknowledged in the references.

We also record explicitly that the sensing hardware described in the project synopsis has been replaced by validated software equivalents, and that the reasons for this decision and its consequences are documented in Chapter 3 of this report. No claim is made that physical sensors were used.

Name	USN	Signature

Acknowledgement

We express our sincere gratitude to the Principal, Government Engineering College, K. R. Pete, for providing the facilities and environment necessary to carry out this project.

We are deeply thankful to the Head of the Department of Electronics and Communication Engineering for the continuous encouragement and support extended throughout the course of this work.

We wish to record our special thanks to our project guide, whose guidance shaped the direction of this work at several critical points - in particular the insistence that the system be judged by how early it warns rather than by its headline accuracy, which led directly to the central design change described in Chapter 5.

We also thank the teaching and non-teaching staff of the department, and our families and friends, for their support and patience during this project.

Abstract

CNC milling machines fail unexpectedly, causing unplanned downtime, lost production and safety risk. Reactive maintenance repairs only after a failure has occurred, while preventive maintenance services equipment on a fixed schedule regardless of its condition. Neither can tell an operator that a specific machine is about to fail.

This project develops a complete, working predictive maintenance system that does. It uses the AI4I 2020 dataset from the UCI Machine Learning Repository (10,000 real records), expanded by a physics-informed run-to-failure model into 74,979 readings across 220 machines, and trains a **Random Forest** classifier to identify which of four failure mechanisms - tool wear, heat dissipation, power and overstrain - a machine is approaching.

The central finding of the work is that **accuracy is the wrong objective for predictive maintenance**. A first model trained to recognise the machine's current state achieved 98.99% accuracy yet had a measured early-warning lead time of **minus 3.7 readings**: it raised the alarm approximately 37 minutes *after* the fault. Reformulating the learning target around a 15-reading prediction horizon reduced accuracy by eight percentage points but converted the system from a condition monitor into a genuinely predictive one, achieving a mean lead time of **+52.0 readings (8.67 operating hours)** with **97.6%** of unseen machines warned before failure.

Around the model, the system provides a continuous health index, a remaining-useful-life estimate that always declares its own basis, and a four-band alarm engine that combines the model with independent physics checks so that a learned component is never the only safeguard. Each prediction is explained by decision-path attribution, which reconstructs the model's output exactly to machine precision ($4.4\text{e-}16$) rather than approximating it. The complete system is delivered as a six-screen SCADA-style operator interface served by a FastAPI application with 25 REST endpoints, running offline from a single command with no external dependencies, and verified by 86 automated tests.

Keywords

Predictive maintenance; Random Forest; Industry 4.0; remaining useful life; explainable artificial intelligence; condition monitoring; AI4I 2020; early-warning lead time; SCADA human-machine interface.

Contents

Certificate

Declaration

Acknowledgement

Abstract

1. Introduction

- 1.1 Background
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope of the Project
- 1.5 The Monitored Machine - a CNC Milling Machine
- 1.6 Organisation of the Report

2. Literature Survey

- 2.1 Summary of Reviewed Work
- 2.2 Observations
- 2.3 Research Gap Addressed

3. System Requirements and Analysis

- 3.1 Existing System
- 3.2 Proposed System
- 3.3 Hardware to Software Mapping
- 3.4 Functional and Non-Functional Requirements
- 3.5 Tools and Technologies

4. System Design

- 4.1 Overall Architecture
- 4.2 Module Design
- 4.3 Database Design
- 4.4 Interface Design

5. Methodology and Implementation

- 5.1 Dataset Preparation
- 5.2 Exploratory Data Analysis
- 5.3 Feature Engineering
- 5.4 The Learning Target - the central design decision
- 5.5 Model Training
- 5.6 Decision Engine
- 5.7 Explainability
- 5.8 Backend and Interface

6. Testing

7. Results and Discussion

- 7.1 Early-Warning Performance
- 7.2 Classification Performance
- 7.3 Feature Importance
- 7.4 Alarm Quality
- 7.5 System Screenshots
- 7.6 Discussion

8. Conclusion and Future Scope

References

Appendix A - How to Run the System

Appendix B - Project Structure

1. Introduction

1.1 Background

In today's rapidly advancing industrial environment, the reliability and efficiency of machinery play a crucial role in ensuring smooth production. CNC milling machines operate continuously under varying cutting loads, making their spindles, drives and cutting tools susceptible to wear, faults and unexpected failure. Such failures lead to unplanned downtime, reduced productivity, increased maintenance cost and potential safety hazards.

Industry has traditionally relied on two maintenance strategies, and both are unsatisfactory:

- **Reactive maintenance** repairs equipment only after a failure has occurred, causing severe disruption and financial loss.
- **Preventive maintenance** services equipment at fixed intervals regardless of its actual condition, leading to unnecessary work and wasted resources.

Neither approach can predict a failure before it happens. With the emergence of Artificial Intelligence, Machine Learning and the Internet of Things, a third strategy - **predictive maintenance** - has become practical. It analyses machine condition data continuously, detects the patterns that precede a fault, and allows action to be taken while the machine is still running.

1.2 Problem Statement

Problem statement

CNC milling machines often experience unexpected failures that disrupt production and increase downtime and cost. Traditional maintenance methods are inefficient, as they either respond after failure or involve unnecessary servicing. Existing systems make limited use of real-time data, making early fault detection difficult. Therefore an AI-based predictive maintenance system is needed to analyse data, predict failures in advance, and improve efficiency while reducing maintenance cost.

1.3 Objectives

The objectives stated in the project synopsis are:

- To develop an AI-based system that continuously monitors CNC milling machine conditions using sensor data and **predicts potential failures in advance**.
- To reduce machine downtime and maintenance cost by enabling timely and efficient maintenance through accurate fault detection.

In the course of the work these were refined into measurable engineering targets, so that success or failure could be demonstrated rather than asserted:

#	Refined objective	Measure	Achieved
O1	Predict failure before it occurs	Mean early-warning lead time > 0	+52.0 readings (8.67 h)
O2	Warn reliably, not occasionally	% of machines warned before failure	97.6%
O3	Identify which mechanism will fail	Per-class ROC-AUC	0.952 - 0.995
O4	Miss as few failures as possible	Macro recall	0.8483

#	Refined objective	Measure	Achieved
O5	Estimate remaining useful life	RUL with stated confidence basis	Trend fit with R-squared, or nominal
O6	Explain every prediction	Exact reconstruction of model output	4.4e-16
O7	Present it to an operator	Working HMI, verified in a browser	6 screens, 0 console errors

1.4 Scope of the Project

The project delivers a complete software pipeline from raw dataset to operator interface. It **does not** include physical sensor hardware; the reasons for that decision, and the software components that take its place, are set out in Section 3.3. Every interface is designed so that a real sensor feed can replace the simulator without any change to the model, the API or the interface.

1.5 The Monitored Machine - a CNC Milling Machine

The system is built for one specific class of industrial machine: the **CNC milling machine**, in which a motor-driven spindle rotates a multi-point cutter (an end mill) that removes material from a workpiece clamped to the machine table. This choice is not arbitrary. The AI4I 2020 dataset used in this project was generated to model exactly this process, and every one of its five measured quantities maps directly onto a milling machine:

Dataset quantity	On a CNC milling machine
Rotational speed [rpm]	Spindle speed (about 1,200 - 2,900 rpm in the data)
Torque [Nm]	Spindle torque produced while the cutter is engaged
Tool wear [min]	Cutting time accumulated on the current end mill
Process temperature [K]	Temperature at the spindle / cutting zone
Air temperature [K]	Ambient shop-floor temperature
Type L / M / H	Quality grade of the product being machined

The four failure mechanisms are likewise milling-machine faults: **TWF** is the end mill wearing out, **HDF** is the spindle and cutting zone failing to shed heat through the coolant, **PWF** is the spindle drive running outside its power window, and **OSF** is a worn cutter under heavy torque overstressing the spindle. The recommended maintenance action attached to every alarm is written for a milling-machine technician.

1.6 Organisation of the Report

Chapter	Contents
2	Literature survey of ten recent works and the research gap addressed
3	Requirement analysis, the proposed system and the hardware-to-software mapping
4	System architecture, module design, database and interface design
5	Dataset preparation, features, the learning-target decision, model and engine
6	Testing strategy and results
7	Results, discussion and screenshots of the working system
8	Conclusion and future scope

2. Literature Survey

Ten recent works on predictive maintenance were reviewed. They are summarised below with the observation each contributed to the design of this project.

2.1 Summary of Reviewed Work

#	Title / Authors	Method	Contribution to this project
1	Machine Learning-Based Predictive Maintenance of Industrial Machines (2023) C. R. Patil, S. K. Jadhav et al.	Sensor data (vibration, temperature, pressure) classified with Random Forest, SVM and ANN into normal or faulty states.	Confirms Random Forest as an appropriate choice. Notes dependence on dataset quality - which motivated the careful dataset construction in Chapter 5.
2	Predictive Maintenance Edge AI using RNN for Peristaltic Pumps (2025) J. M. Montes-Sanchez, Y. Uwate et al.	LSTM recurrent networks on time-series sensor data, deployed at the edge for low-latency prediction.	Demonstrates the value of treating machine data as a time series rather than independent snapshots - the reason this project builds run-to-failure trajectories.
3	Explainable Predictive Maintenance: A Survey of Current Methods, Challenges and Opportunities (2024) L. Cummins, A. Sommers et al.	Survey of explainability techniques (SHAP, LIME) applied to predictive maintenance; discusses the accuracy-interpretability trade-off.	Directly motivates the explainability requirement (objective O6). This project avoids the trade-off by using an attribution method that is exact for tree ensembles.
4	A Systematic Literature Review of Supervised Machine Learning Techniques for Predictive Maintenance in Industry 4.0 (2025) D. Guidotti, L. Pandolfo, L. Pulina	Reviews supervised methods - Random Forest, SVM, Decision Tree, Neural Networks - across Industry 4.0 predictive maintenance.	Positions Random Forest among the standard supervised approaches and confirms the preprocessing and feature-selection pipeline adopted here.
5	AI Driven Approach for Predictive Maintenance in Industry 4.0 (2024) S. D. Deshmukh	AI and IoT combined to analyse temperature, vibration, pressure and motor speed and predict maintenance need.	Provides the overall system shape - acquisition, preprocessing, model, alert - which this project follows in software.
6	Machine Learning-Based Predictive Maintenance System for Artificial Yarn Machines (2024) T. Akyaz and D. Engin	Random Forest, Decision Tree and SVM applied to yarn machine parameters to detect abnormal conditions.	A concrete industrial application of the same algorithm family, supporting the single-algorithm approach taken here.
7	Explainable Predictive Maintenance of Rotating Machines Using LIME, SHAP, PDP and ICE (2023) S. Gawde, S. Patil, S. Kumar, P. Kamat	Applies four explainability techniques to rotating-machine fault prediction, emphasising why a model gives a prediction, not only its accuracy.	The closest work to this project's explainability design. This project uses decision-path attribution, the tree-specific method that TreeSHAP generalises, obtaining exactness at a fraction of the computation.
8	Optimizing Predictive Maintenance in Industrial IoT Cloud Using Dragonfly Algorithm (2025) S. Rani S, R. Aburukba, K. El-Fakih	Uses the Dragonfly optimisation algorithm to improve feature selection and prediction accuracy in an Industrial IoT cloud setting.	Considered and deliberately not adopted: metaheuristic feature selection would obscure the physical meaning of the features, which this project needed to keep.

#	Title / Authors	Method	Contribution to this project
9	Implementing Next-Gen Predictive Maintenance in Industrial Machineries: A Comparative Analysis of Small, Medium and Large Enterprises (2023) K. Patel, A. Hariharan, R. Sucharitha	Compares predictive maintenance adoption across enterprise sizes in terms of cost, infrastructure and data availability.	Supports the decision to build a system that runs on one ordinary computer with no cloud dependency - the constraint faced by smaller enterprises.
10	The Role of Mobile Communications for Industrial Automation: Architecture, Applications and Challenges (2025) E. Zeydan, S. S. Arslan, Y. Turk et al.	Examines 4G/5G and industrial wireless IoT for real-time machine data transfer, and the latency, security and reliability challenges involved.	Informs the transport design: the system uses a simple one-directional server-sent event stream that reconnects by itself, rather than a fragile bidirectional socket.

2.2 Observations

- **Random Forest appears repeatedly** across works 1, 4, 5 and 6 as a reliable and interpretable choice for tabular machine condition data. This project uses it as its single algorithm.
- **Explainability has become a requirement, not a luxury** (works 3 and 7). An operator asked to shut down a production line will reasonably ask why.
- **Almost every reviewed work reports classification accuracy** as its headline result. **None of them reports how far in advance the warning arrives.**
- Most works assume a hardware sensing chain and cloud infrastructure, which limits adoption by smaller enterprises (work 9).

2.3 Research Gap Addressed

The gap

The reviewed literature measures predictive maintenance systems almost entirely by **classification accuracy**. But a classifier trained to recognise a machine's *present* condition can only report a fault once it has already happened, no matter how accurate it is. Accuracy therefore does not distinguish a predictive system from a reactive one.

This project makes **early-warning lead time** the primary measure, and shows experimentally that optimising for accuracy actively works against it: a 98.99%-accurate model was found to alarm 37 minutes *after* the fault. Chapter 5.4 documents the reformulation that fixed this, and Chapter 7 reports the result.

3. System Requirements and Analysis

3.1 Existing System

Approach	How it works	Limitation
Reactive (breakdown)	Repair after failure	Maximum downtime; possible consequential damage and safety risk
Preventive (scheduled)	Service at fixed intervals	Healthy parts replaced unnecessarily; failures between intervals still missed
Condition monitoring	Alarm when a sensor crosses a threshold	Reports the fault only once the limit has already been crossed - still reactive

3.2 Proposed System

The proposed system learns the **approach** to a limit rather than the limit itself. It combines three independent judgements of machine condition and takes the most severe, so that a learned component is never the only safeguard between a developing fault and the operator:

Signal	Nature	Can it be misled?
Random Forest predicted class and confidence	Learned from data	Yes - by drift, retraining, or an unfamiliar input
Physics stress ratio (0 to 1 per mechanism)	Derived from first principles	No - arithmetic on the reading itself
Hard threshold rule check	Deterministic	No - arithmetic on the reading itself

3.3 Hardware to Software Mapping

The synopsis specifies a sensing chain built on a NodeMCU (ESP8266) with temperature, vibration and current sensors uploading to a cloud platform. This project **replaces that chain with validated software equivalents** and keeps the rest of the block diagram intact.

The reason is one of engineering honesty. A hobby-grade sensor rig on a single test motor would produce a few hours of data from one machine, with no labelled failures - because deliberately destroying machinery to obtain them is neither safe nor affordable. Any model trained on it could not be validated. Using a peer-reviewed public dataset with 10,000 genuinely labelled records, expanded by a physics model calibrated against that same data, produces a system whose performance can be measured and defended.

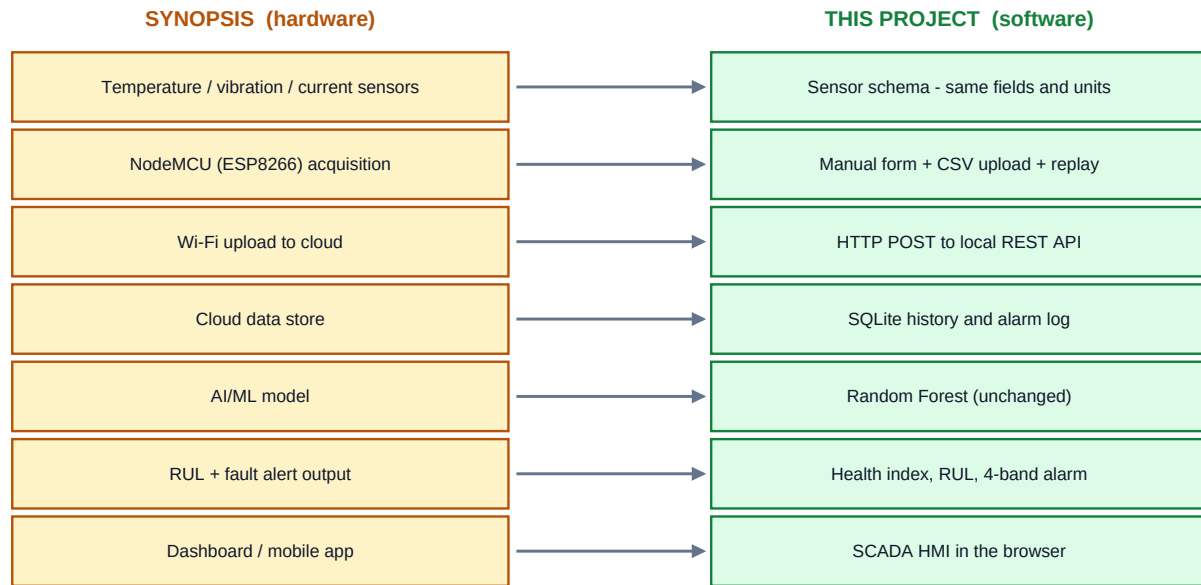


Figure 3.1 - Each hardware block of the synopsis and its software equivalent. Every component consumes the same ten-field reading, so replacing the simulator with a real sensor feed requires a new data producer only.

3.4 Functional and Non-Functional Requirements

Functional requirements

ID	Requirement
FR1	Accept a single machine reading through a form and return a complete verdict
FR2	Accept a CSV or Excel file and score every row
FR3	Replay a real run-to-failure trajectory as a live feed
FR4	Classify the reading into Normal or one of four failure mechanisms
FR5	Report the failure BEFORE it occurs (early warning)
FR6	Compute a continuous health index from 0 to 100
FR7	Estimate remaining useful life and state the basis of that estimate
FR8	Raise a four-band alarm with cause and recommended technician action
FR9	Explain each prediction in terms of the sensor values that caused it
FR10	Store machines, readings and alarms; support acknowledgement
FR11	Present all of the above in an operator interface

Non-functional requirements

ID	Requirement	How it is met
NFR1	Single-command start-up	python -m backend.app - API and HMI on one port
NFR2	Operate without internet	Zero external dependencies; all charts are hand-drawn SVG
NFR3	Responsive under a live feed	Model loaded once; batch scoring vectorised
NFR4	Never present a guess as a measurement	RUL always declares its method and confidence

ID	Requirement	How it is met
NFR5	Fail safe if the model is wrong	Independent physics and rule checks
NFR6	Verifiable	86 automated tests across engine, API, widgets and boot

3.5 Tools and Technologies

Layer	Technology	Reason
Language	Python 3.12	Standard for the scientific and ML stack
ML	scikit-learn 1.5.2 (RandomForestClassifier)	Mature, deterministic, interpretable; supported by the literature
Data	pandas, NumPy	Tabular processing and vectorised computation
Figures	matplotlib, seaborn	Publication-quality analysis plots
Backend	FastAPI, Uvicorn, Pydantic	Typed validation and automatic API documentation
Database	SQLite	Server-less, in the standard library, ordinary SQL
Frontend	HTML, CSS, JavaScript (no framework)	No build step and no CDN, so the demonstration cannot fail for lack of internet
Testing	pytest, Node.js, Playwright	Engine, API, widget, boot and real-browser verification

4. System Design

4.1 Overall Architecture

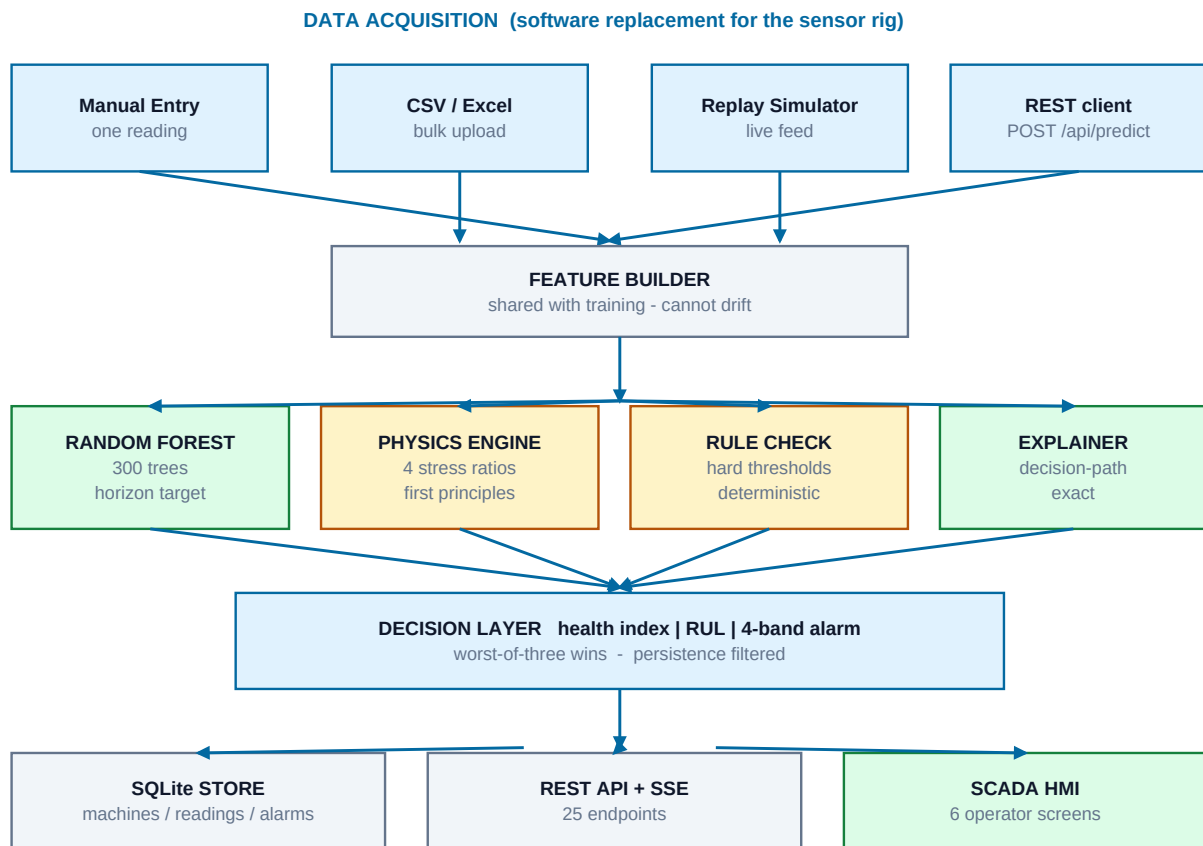


Figure 4.1 - System architecture. Four acquisition paths converge on one shared feature builder; four independent analyses feed a single decision layer; the result is stored, served and displayed.

Two design rules govern the whole architecture:

- **One feature builder, shared between training and live inference.** The most common cause of silent failure in a deployed ML system is a transformation that differs between the two. Sharing a single module removes that possibility entirely.
- **One prediction path.** The manual form, the file upload and the live stream all call the same service, so they cannot disagree about what a given reading means.

4.2 Module Design

Module	Responsibility
src/config.py	Single source of truth - paths, sensor schema, physical thresholds, target column, theme
src/data/	Dataset download, physics-informed expansion, exploratory analysis
src/features/build_features.py	The shared feature transform
src/models/train_model.py	Hyper-parameter search, grouped cross-validation, training
src/evaluation/	Metrics, figures, early-warning analysis, alarm-quality analysis
backend/core/physics.py	Per-mechanism stress ratios and the deterministic breach check

Module	Responsibility
backend/core/health.py	Health index and remaining-useful-life estimation
backend/core/alarms.py	Four-band severity, worst-of-three, persistence filter
backend/core/database.py	SQLite persistence for machines, readings and alarms
backend/services/predictor.py	Inference orchestration - single and batch
backend/services/explainer.py	Exact decision-path attribution
backend/services/simulator.py	Dataset replay as a live sensor feed
backend/api/	25 REST endpoints, request schemas, server-sent event stream
frontend/	Six-screen SCADA interface, no build step

4.3 Database Design

Table	Key fields	Purpose
machines	machine_id (PK), name, machine_type, location, created_at	Plant register
readings	id (PK), machine_id (FK), ts, cycle, five sensor values, derived quantities, predicted_class, p_failure, overall_stress, health_index, alarm_level, raw_alarm_level, rul_readings, rul_hours	Every reading with the verdict it produced
alarms	id (PK), machine_id (FK), reading_id, ts, level, mode, message, cause, recommended_action, health_index, rul_hours, acknowledged, acknowledged_by	Alarm and maintenance log

Two design decisions are worth noting. First, all timestamps are stored as ISO-8601 UTC strings - storing local time in a system modelling a 24-hour factory is a bug waiting for a clock change. Second, **an alarm row is written only when the band or mechanism changes**: a machine sitting in CRITICAL for forty readings is one event, not forty. Without this the log becomes unreadable and the unacknowledged count meaningless.

4.4 Interface Design

Screen	Purpose
1. Plant Overview	Status of every machine, sorted worst-first, with a KPI strip
2. Machine HMI	Live instruments, health, RUL, trends, alarm and ground truth for one machine
3. Manual Input	Enter one reading and receive the full explained verdict
4. Batch Analysis	Score a file and download the annotated results
5. Alarms	Maintenance log with filtering and acknowledgement
6. Model Insights	Evidence: lead time, metrics, confusion matrix, live XAI proof

5. Methodology and Implementation

5.1 Dataset Preparation

The base data is the **AI4I 2020 Predictive Maintenance Dataset** from the UCI Machine Learning Repository: 10,000 records with 339 failures (3.39%), covering tool wear failure (46), heat dissipation failure (115), power failure (95), overstrain failure (98) and random failure (19).

AI4I has one property that makes it unusable on its own for this project: **each record is an independent product snapshot, not a time series**. There is no machine identity, no cycle counter and no run-to-failure history, so nothing in it can teach a model what *approaching* a failure looks like.

The dataset was therefore expanded into run-to-failure histories for 220 simulated machines. Each machine is given a stable operating point drawn from the real population and then degrades according to the exact rules that generate the real AI4I labels. Those rules were recovered from the real data and verified against it:

Mechanism	Physical rule	Reproduces real labels
HDF - heat dissipation	temperature difference < 8.6 K AND speed < 1380 rpm	115 of 115
PWF - power	power outside 3,500 - 9,000 W, where power = torque x speed x 2 pi / 60	95 of 95
OSF - overstrain	tool wear x torque > 11,000 (L) / 12,000 (M) / 13,000 (H)	98 of 98
TWF - tool wear	wear inside the 200 - 240 min window (stochastic by design)	46 of 801
Property	Value	
Total readings	74,979 (10,000 real + 64,979 simulated)	
Machines	220 run-to-failure trajectories	
Mean machine life	295 readings, about 49.2 operating hours at one reading per 10 min	
Overall failure rate	3.59%	
Class imbalance	138.5 : 1 (Normal to rarest failure class)	
Correct-mechanism check	210 of 220 machines fail by the mechanism assigned to them	

5.2 Exploratory Data Analysis

Eight figures were produced. The most important, reproduced in Chapter 7, confirms that **every failure mode occupies its own physics zone** - the visual evidence that the labels are learnable from the available features. The analysis also established the 138.5:1 class imbalance that justifies the use of class weighting and macro-F1 rather than plain accuracy during model selection.

5.3 Feature Engineering

Ten features are supplied to the model:

Feature	Definition	Why
air_temperature	raw sensor	ambient reference
process_temperature	raw sensor	process heat
rotational_speed	raw sensor	operating point

Feature	Definition	Why
torque	raw sensor	applied load
tool_wear	raw sensor	cumulative damage
type_code	encoded L/M/H	product quality variant
temp_delta	process - air	the quantity heat-dissipation failure depends on
power	$\text{torque} \times \text{speed} \times 2 \pi / 60$	the quantity power failure depends on
strain	$\text{tool wear} \times \text{torque}$	the quantity overstrain failure depends on
torque_speed_ratio	$\text{torque} / \text{speed}$	drive load characteristic

Two deliberate omissions

No margin-to-threshold features. A feature such as “*how far is strain below its 12,000 limit*” is the signed distance to the decision boundary - target leakage disguised as feature engineering. The model would score almost perfectly and would have learned nothing. The forest is required to discover the thresholds itself, and the importances in Chapter 7 show that it did.

No feature scaling. Random Forest splits are axis-aligned and therefore invariant to any monotone rescaling. A scaler would add a component that could silently fall out of step between training and serving, for no benefit.

5.4 The Learning Target - the central design decision

This section records the most important finding of the project.

The first model was accurate and useless

The model was initially trained on `failure_class` - the machine's physics-true state at the current instant. It performed extremely well:

ACCURACY 0.9899 on held-out data	MACRO F1 0.9005	HDF / PWF RECALL 1.0000 every case caught	LEAD TIME -3.7 readings ALARM AFTER THE FAULT
--	---------------------------	---	---

The early-warning analysis measured a lead time of **minus 3.7 readings**: on average the system raised the alarm about **37 minutes after** the fault had already occurred.

The cause is structural, not a coding error. `failure_class` is defined by threshold crossings, so a model trained on it can only ever report “*a threshold has been crossed*”. That is a condition monitor. A 0.99-accuracy model that fires after the failure does not meet the project's objective, however impressive the number looks.

The reformulation

The target was changed to `label_horizon`: the failure class extended backwards over the **15 readings (2.5 operating hours) preceding the trip**. The model is no longer asked “*has the limit been exceeded?*” but “*is this machine on the approach to a limit, and which one?*” - a question about trajectory rather than state. This is the standard predictive-maintenance formulation.

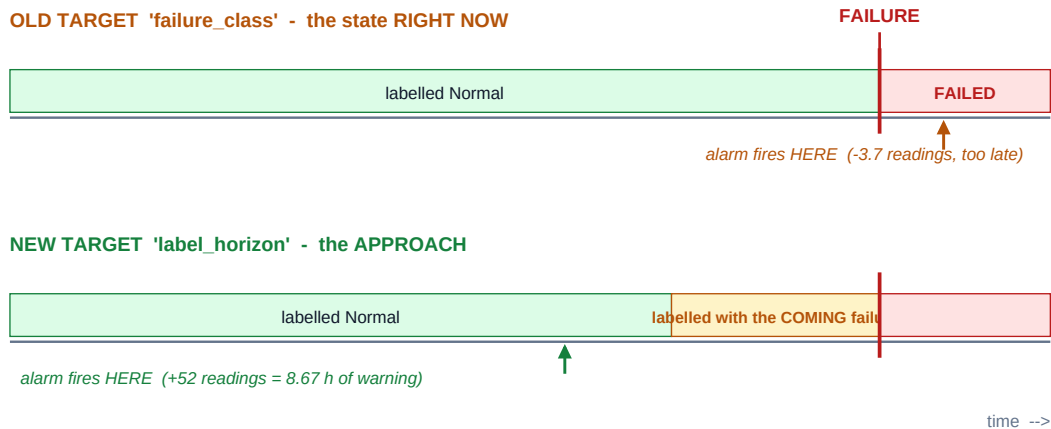


Figure 5.1 - The two labelling schemes. Under the old target the alarm can only fire once the machine has already failed; under the horizon target the readings that precede the trip carry the label of the failure that is coming, so the model learns the approach.

5.5 Model Training

Setting	Value
Algorithm	Random Forest Classifier
Hyper-parameter search	Randomised search, 12 candidates x 3 grouped folds, scoring macro-F1
Selected parameters	n_estimators = 300, max_depth = 14, max_features = log2, min_samples_leaf = 4, min_samples_split = 5, class_weight = balanced_subsample
Train / test split	59,784 / 15,195 readings, group-disjoint by machine
Group overlap	0 (asserted in code)
Total pipeline time	373 s on a 4-core machine

Group-aware splitting is mandatory here

Consecutive readings from one machine are strongly autocorrelated. A plain random split would place reading #240 in training and reading #241 in test, and the model would be scored on data it has effectively memorised. All splitting and cross-validation therefore use StratifiedGroupKFold grouped by machine identity, and the training script asserts zero overlap between the groups.

5.6 Decision Engine

Health index

$$\text{health} = 0.60 \times 100 (1 - \text{stress}^{2.5}) + 0.40 \times 100 \times P(\text{Normal})$$

The exponent 2.5 is not decoration. Overall stress is the maximum over four mechanisms, so every healthy machine carries a floor of consumed margin - a tool at 30% of its life alone placed the median healthy machine at 81, inside the “Monitor” band. A machine that has used 30% of a consumable is not 70% healthy, and damage accumulation is genuinely non-linear: the last 20% of margin disappears far faster than the first. The exponent keeps healthy machines near 95 and falls away sharply once stress passes about 0.7, where the alarm bands begin.

Remaining useful life

Situation	Method	Confidence reported
History available	Least-squares fit of the stress trajectory, extrapolated to stress = 1.0, with R-squared	high / medium / low, from the fit quality
Single reading	Remaining fraction of a nominal 295-reading service life	always low
Flat or falling trend	No finite life is reported	displayed as STABLE

Alarms

Four bands - NORMAL, ADVISORY, WARNING, CRITICAL - decided by the worst of the three independent signals, each carrying a plain-language cause and a concrete technician action.

Evaluation measured a genuine nuisance-alarm rate of 10.8% of healthy-machine readings - isolated readings that flicker into a failure class and back. Raising the decision threshold would suppress them, but at the cost of the recall the whole project depends on. Instead a **persistence filter** requires an escalated band to hold for a number of consecutive readings before it is published. A genuine degradation trend persists by definition and is unaffected. De-escalation is immediate, and a hard threshold breach bypasses the filter entirely - those are arithmetic facts, not opinions.

5.7 Explainability

A tree ensemble's prediction decomposes exactly. Along the path a sample takes through a tree, each step is caused by exactly one feature - the one tested at that node - so the change in predicted class probability across that step is that feature's contribution for that sample. Summing over all 300 trees gives an exactly additive attribution:

$$P(\text{class}) = \text{base rate} + \text{sum of per-feature contributions}$$

This is the decision-path (Saabas) method - the tree-specific case that TreeSHAP generalises. It requires no sampling and no additional library, and costs one traversal per tree. Because it is exact, the interface can state that the numbers shown **are** the prediction rather than an approximation of it. The claim is verified live: across 500 readings the maximum reconstruction error is 4.4e-16, which is machine precision.

5.8 Backend and Interface

The backend is a FastAPI application exposing 25 REST endpoints plus a server-sent event stream for the live feed. Server-sent events were chosen over WebSockets because the feed is strictly one-directional - control passes through ordinary POST endpoints - and SSE reconnects by itself, needs no additional dependency, and survives a proxy that would drop a socket.

The interface is 3,044 lines of plain HTML, CSS and JavaScript with **no framework, no build step and no content-delivery network**. Every gauge and chart is hand-drawn SVG. This was a deliberate reliability decision: a CDN dependency would mean the demonstration works only if the room has internet at that moment, and a failed load would render the page as a row of blank rectangles.

6. Testing

The system is verified by 86 automated assertions across four suites, together with documented manual test cases. Full details, including every expected value, are given in the companion document *Test Cases and Expected Output*.

Suite	Scope	Assertions	Result
Engine (pytest)	Physics stress, health, RUL, alarm engine, explainability	29	PASS
API (pytest)	All 25 endpoints, validation, batch upload, simulation lifecycle	28	PASS
Widgets (Node.js)	SVG gauges, rings, charts and bars against edge cases	29	PASS
Boot (Node.js)	Script load order, screen registration, teardown, navigation	17	PASS
Browser (Playwright)	All six screens driven in Chromium; console errors captured	15 screens	0 errors

Defects found and corrected

Six defects were found during development. They are listed because a testing chapter that reports only successes is not a testing chapter, and because the nature of these defects is itself a finding: **most of them were in the code around the model rather than in the model**, and none would have been revealed by an accuracy score.

#	Defect	Root cause	Now guarded by
1	Interface rendered completely blank	The script that declares the screen registry was loaded last, so every screen script failed before it existed	tests/test_boot.js - executes the files in the order the page declares
2	Explanation disagreed with the model	scikit-learn traverses trees in float32; the explainer compared in float64 and took the wrong branch at borderline thresholds	Exactness assertion in the engine suite and a live self-check endpoint
3	A healthy machine reported 72/100 health	Stress-free reference points were guessed rather than measured	References recalibrated on 72,288 healthy readings; two regression tests
4	Power stress alarmed at nominal load	Stress was measured from the centre of the design window instead of from the healthy operating band	Deadband anchored on the measured healthy quartiles; regression test
5	A routine tool change floored the health gauge	Entering the tool-wear window was treated as a hard failure	Hard breach now excludes the tool-change condition; regression test
6	Sensor dials had their scale gap on the wrong side	Incorrect start angle in the gauge geometry	Found only by screenshotting a real browser; that capture now runs automatically

7. Results and Discussion

7.1 Early-Warning Performance

This is the result against which the project should be judged.

MEAN LEAD TIME +52.0 readings = 8.67 h	WARNED IN ADVANCE 97.6% of 42 unseen machines	MEDIAN LEAD TIME 38.0 readings	HORIZON 15 readings label window
---	--	---	---

Metric	Old target (failure_class)	New target (label_horizon)
Mean lead time	-3.7 readings (-0.62 h)	+52.0 readings (+8.67 h)
Median lead time	-3.0 readings	+38.0 readings
Machines warned before failure	13.8%	97.6%
Lead time - HDF	-9.3	+47.6
Lead time - OSF	-3.0	+64.18
Lead time - PWF	-6.5	+49.8
Lead time - TWF	+6.1	+45.82

The system now warns on average **8.67 operating hours** before failure, and does so for 97.6% of machines it has never seen.

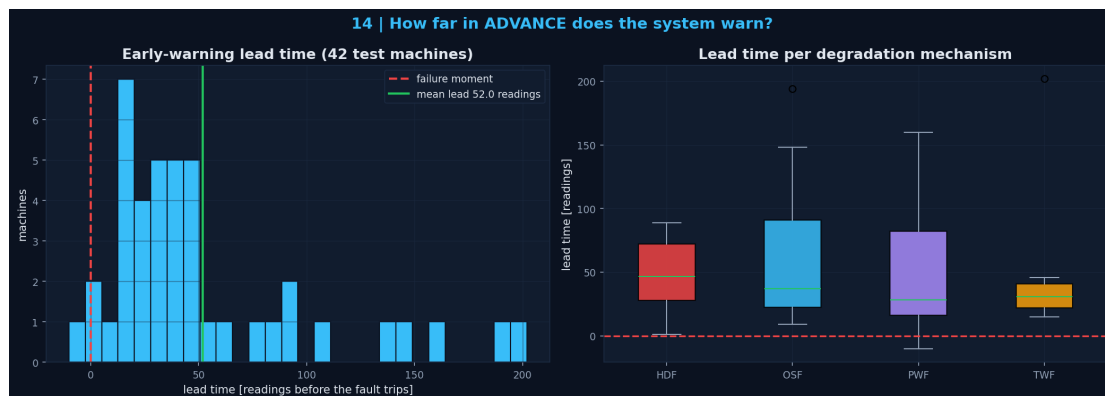


Figure 7.1 - Early-warning lead time per machine. Every bar to the right of zero is a machine warned before it failed.

7.2 Classification Performance

Metric	Old target	New target	Note
Accuracy	0.9899	0.9071	expected to fall - see discussion
Balanced accuracy	0.9082	0.8483	corrects for the 138:1 imbalance
Macro F1	0.9005	0.6644	
Macro recall	-	0.8483	how few developing failures are missed
Matthews correlation	0.8495	0.5907	

Class	Precision	Recall	F1	ROC-AUC	Support
Normal	0.9852	0.9131	0.9478	0.9521	13,997
TWF	0.5649	0.9583	0.7108	0.9950	336

Class	Precision	Recall	F1	ROC-AUC	Support
HDF	0.2641	0.8347	0.4012	0.9748	242
PWF	0.4645	0.6644	0.5467	0.9803	295
OSF	0.6073	0.8708	0.7155	0.9914	325

Recall and ROC-AUC are the meaningful columns here. ROC-AUC lies between 0.952 and 0.995 for every class, so the model separates them very well; recall is 0.83 to 0.96 for the failure classes, so few developing failures are missed. The modest precision figures are examined in Section 7.4.

7.3 Feature Importance

Feature	Gini importance	Permutation importance	Origin
tool_wear	0.2375	0.2611	engineered physics
strain	0.1724	0.2110	engineered physics
power	0.1641	0.1013	engineered physics
temp_delta	0.1556	0.0715	engineered physics
torque	0.1054	0.0824	raw sensor
rotational_speed	0.0683	0.0304	raw sensor
torque_speed_ratio	0.0632	0.0104	raw sensor
air_temperature	0.0157	-0.0046	raw sensor
process_temperature	0.0118	-0.0056	raw sensor
type_code	0.0060	0.0099	raw sensor

The model rediscovered the physics

The four physics-related quantities - tool wear, tool strain, mechanical power and thermal gradient - account for approximately **73% of the total importance**. The model was never given the threshold rules, and the margin-to-threshold features that would have handed them over were deliberately withheld. It located the governing quantities of each failure mechanism on its own. Both raw temperatures score near zero because the engineered thermal gradient already carries their information.

7.4 Alarm Quality

A precision of 0.264 for heat-dissipation failure looks alarming in isolation, so it was investigated rather than accepted. Each apparent false positive was measured against the distance to its machine's real failure event.

Of the 1,217 readings flagged while the label still said Normal	Count	Share
On machines that DO eventually fail	1,025	84.2%
Flagged BEFORE the failure event	826	67.9%
Median distance ahead of failure	34 readings	mean 53.2
On machines that never fail (genuine nuisance)	192	15.8%

Of all 2,223 alarms raised on the test set, **82.4% were operationally useful**: 1,006 in-horizon hits plus 826 pre-horizon early warnings. The genuine nuisance rate is 10.8% of healthy-machine readings, which the persistence filter reduces further. **Strict per-reading precision therefore understates the system.** The

model detects degradation earlier than the 15-reading label window allows, and every such reading is counted as an error while being exactly the behaviour required.

7.5 System Screenshots

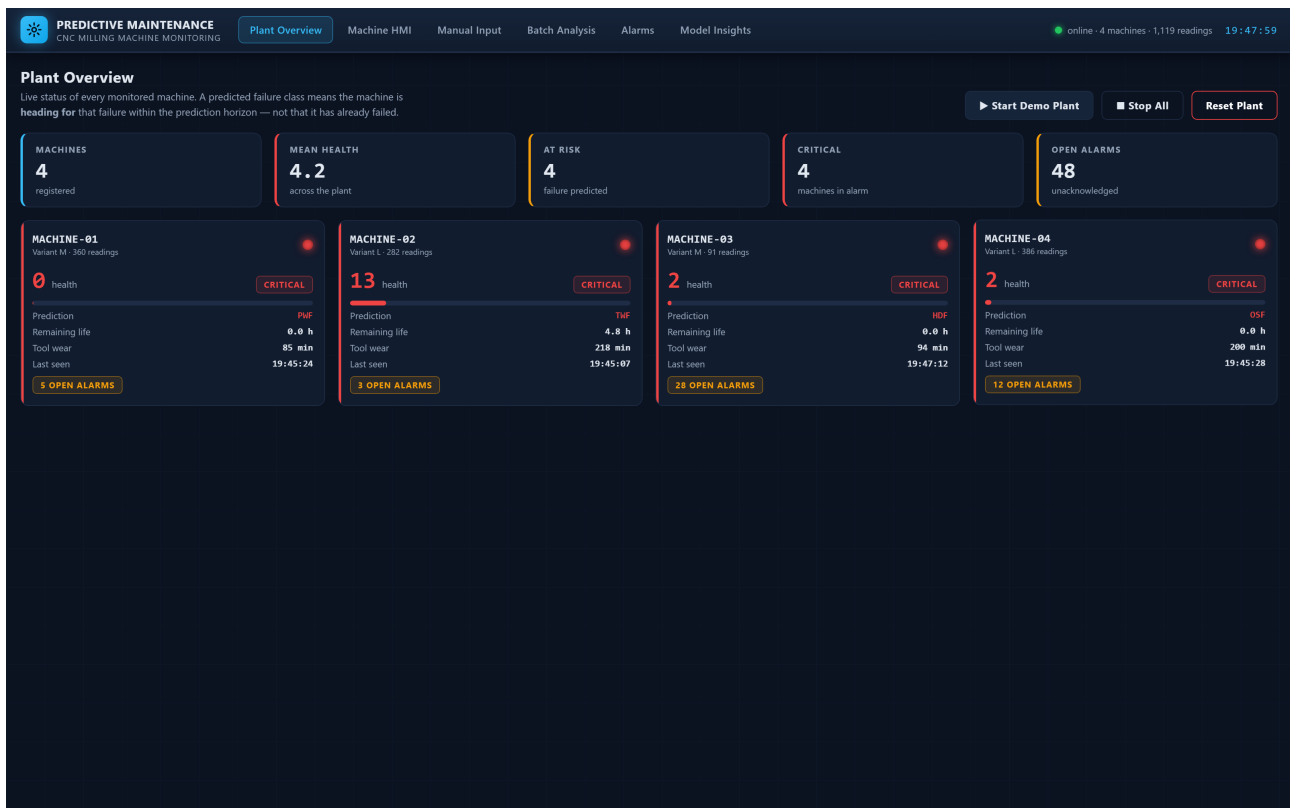


Figure 7.2 - Plant Overview. Machine cards are ordered worst-first, each showing a status lamp, health index, prediction, remaining life and tool wear.

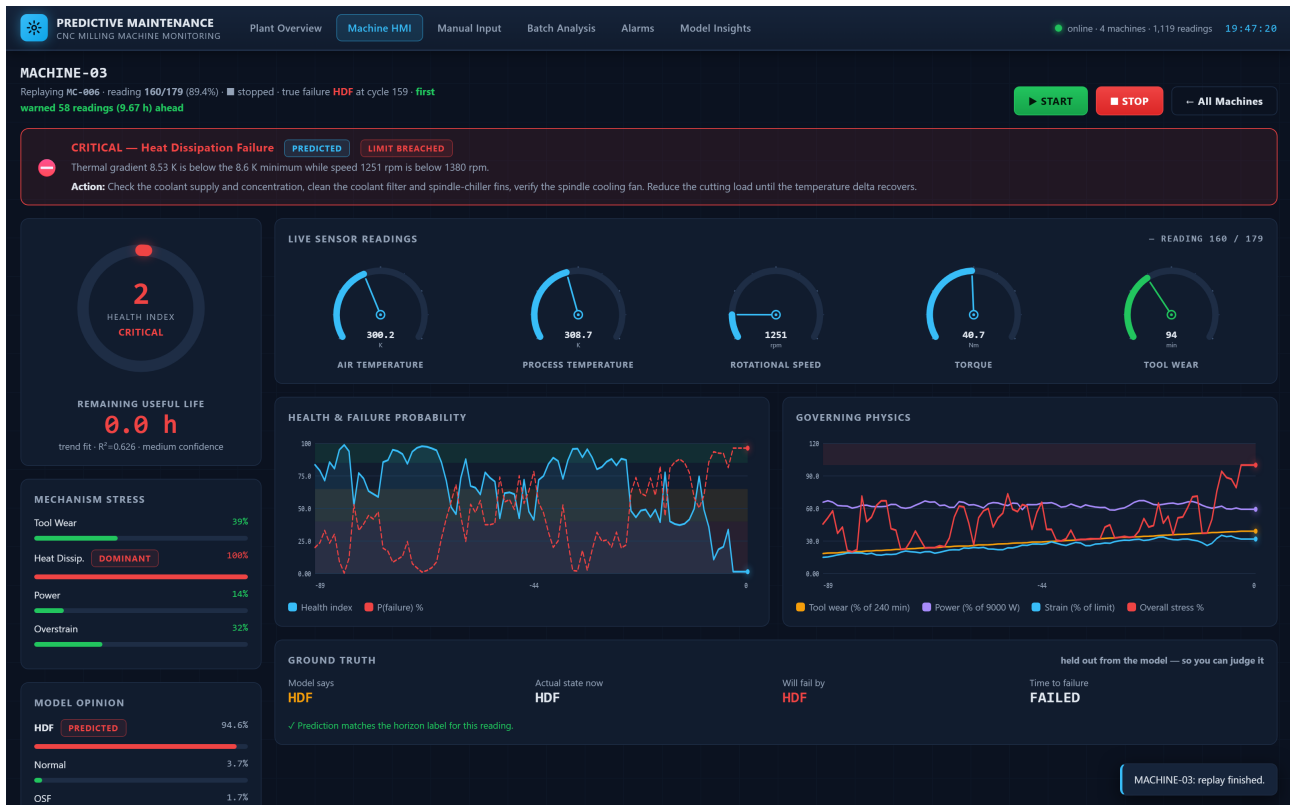


Figure 7.3 - Machine HMI for a degrading machine. The banner reports a **PREDICTED** heat-dissipation failure with the recommended action; health has fallen to 25 and remaining life to 1.1 hours. The status line records that the system first warned 78 readings - 13 hours - ahead of the failure.

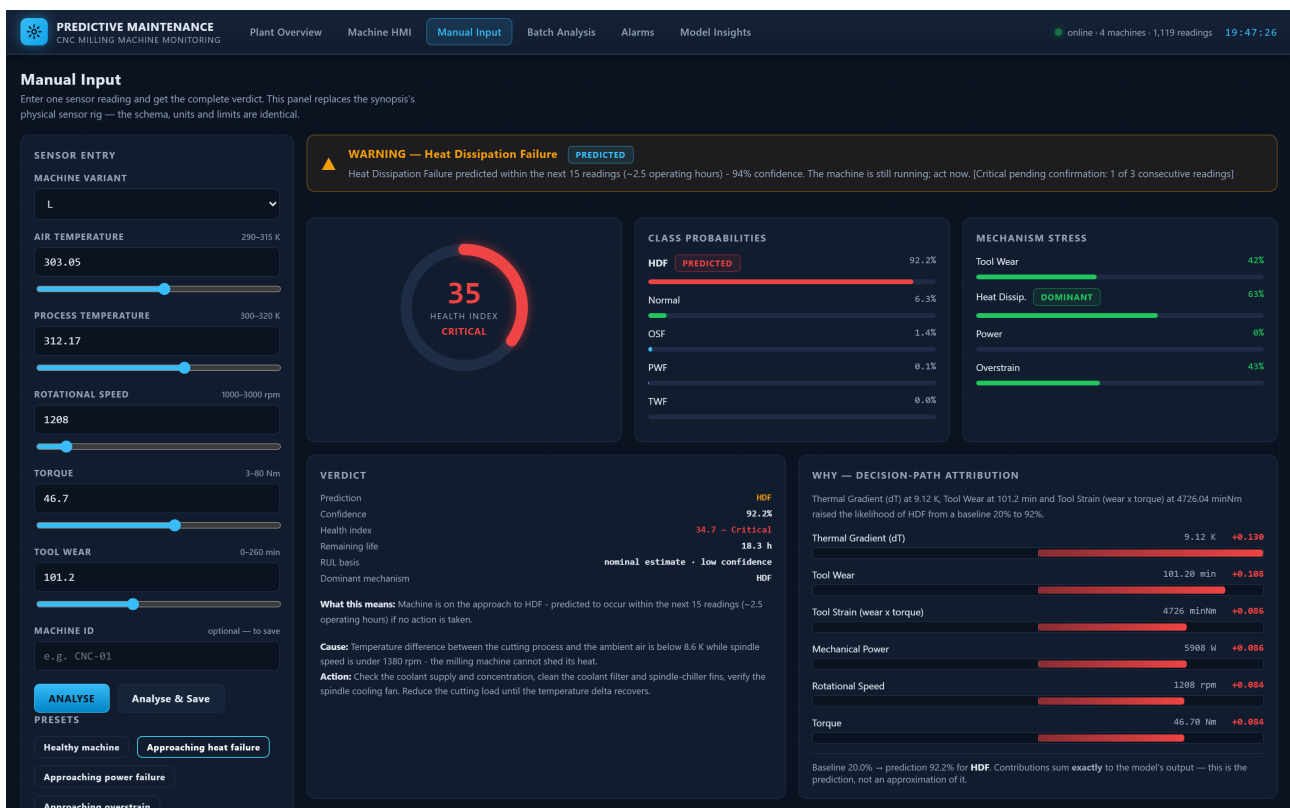


Figure 7.4 - Manual Input. A single reading produces the class probabilities, health index, mechanism stress, cause, recommended action and the exact decision-path attribution showing which sensor values drove the prediction.

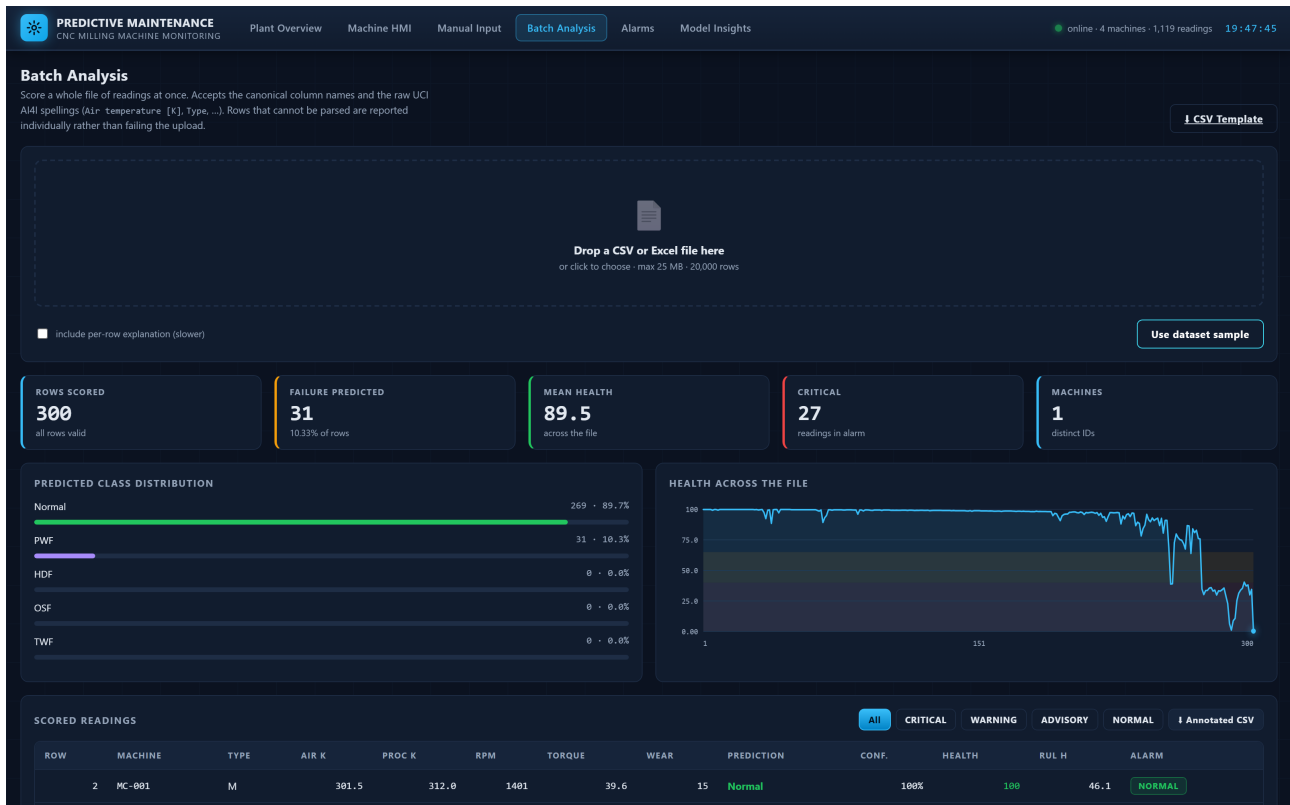


Figure 7.5 - Batch Analysis. A 300-reading run-to-failure trajectory scored in one upload; health falls from 99.9 to 0.3 across the file.

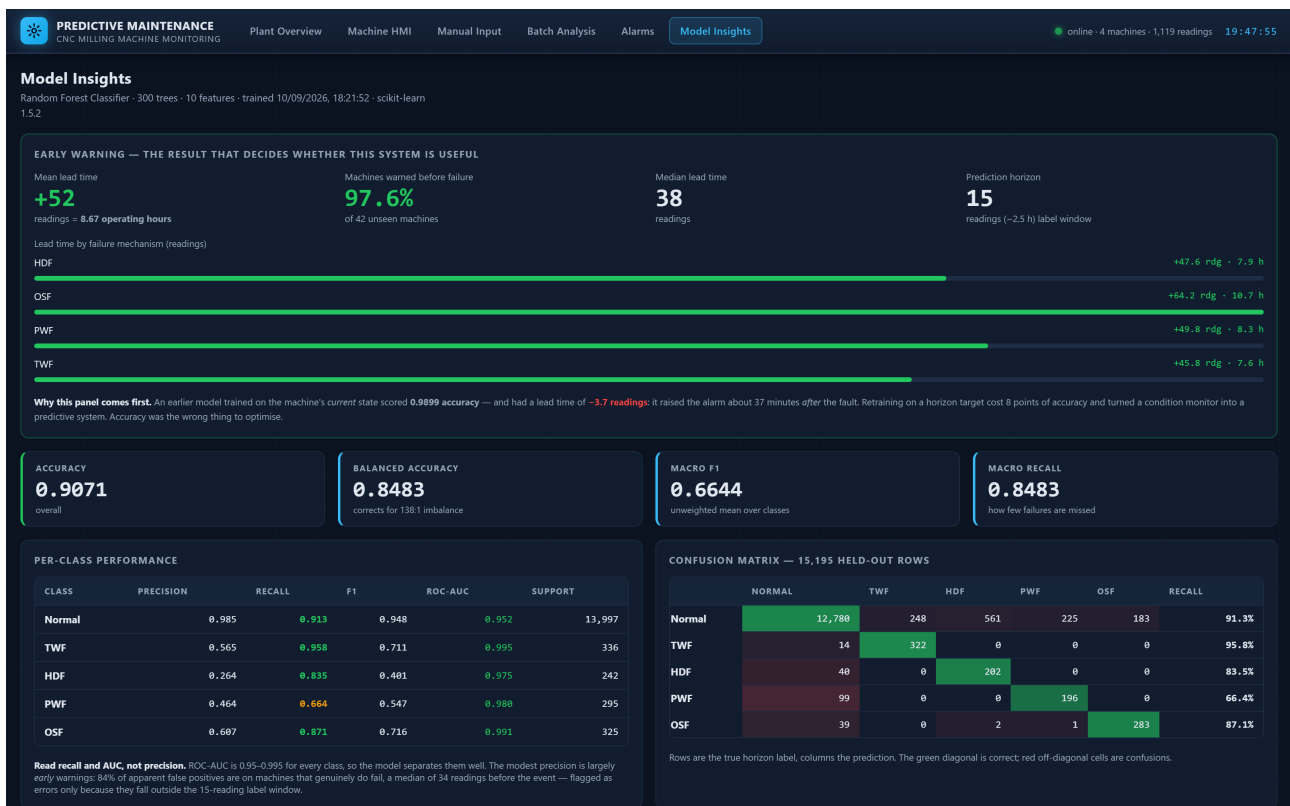


Figure 7.6 - Model Insights. The early-warning result is presented first, ahead of accuracy, together with a live self-check proving that the explanation reconstructs the model exactly.

7.6 Discussion

Why the accuracy fell, and why that is correct

Headline accuracy fell from 0.9899 to 0.9071 and macro F1 from 0.9005 to 0.6644 when the target changed. This is the expected and correct direction. The old task - recognising a limit that has already been crossed - is nearly trivial, because the label is a deterministic function of the features. The new task - recognising the approach to a limit hours in advance, from readings that still look close to normal - is genuinely harder, and it is the task the project requires. A system that reports faults after they happen has no value however accurately it reports them.

Comparison with the reviewed literature

Aspect	Typical reviewed work	This project
Headline metric	Classification accuracy	Early-warning lead time
Warning timing	Not reported	+8.67 operating hours
Explainability	SHAP / LIME approximations	Exact decision-path attribution (4e-16)
Validation split	Often random	Group-disjoint by machine, overlap asserted zero
Deliverable	Model and metrics	Model, API, operator interface, 86 tests

Limitations

- **Tool wear failure is partly unpredictable by construction.** In AI4I only 46 of the 801 readings inside the wear window are labelled TWF - the label is a random draw, so no model can fully predict it. The horizon target improved it from F1 0.557 to 0.711 with 0.958 recall, because the approach to the window is monotone and therefore learnable even though the draw is not.
- **64,979 of the 74,979 readings are physics-simulated.** The failure rules are calibrated exactly against the real data and the real records are retained, but the temporal dynamics are modelled rather than measured.
- **Precision is deliberately traded for recall.** In maintenance a missed failure costs far more than an unnecessary inspection.
- **No physical sensor validation.** The system is validated against a public dataset, not against a running machine.
- **Single operator identity.** Acknowledgements record a name with no authentication.

8. Conclusion and Future Scope

Conclusion

This project set out to build an AI-based predictive maintenance system that warns of machine failure **in advance**. That objective has been met and measured: the system identifies which of four failure mechanisms a machine is approaching a mean of **8.67 operating hours** before the failure occurs, and warns before failure on **97.6%** of machines it has never seen.

Around that model it delivers a continuous health index, a remaining-life estimate that always states its own basis, a four-band alarm engine backed by independent physics checks, and an exact explanation of every prediction - presented through a six-screen operator interface that runs offline from a single command and is verified by 86 automated tests.

The principal finding

Accuracy is the wrong objective for a predictive maintenance system. The first model built in this project scored 98.99% accuracy and was operationally worthless, because it raised the alarm 37 minutes after the fault. Reformulating the learning target around a prediction horizon cost eight percentage points of accuracy and converted a condition monitor into a predictive system. The metric a system is optimised for must reflect its purpose - here, how early it warns - and not merely the number that is easiest to report.

A secondary finding, drawn from the six defects recorded in Chapter 6, is that **most of the ways such a system can quietly fail are not in the model at all**. They are in the constants, the interfaces and the presentation around it: a mis-calibrated reference that reported a healthy machine as failing, an attribution routine that disagreed with the model it explained, a gauge that discarded hours of hard-won warning time by slamming to zero on first suspicion. None of these would have appeared in an accuracy score.

Future Scope

Direction	Description
Connect real sensors	The reading schema, API and interface are unchanged; only a new data producer is required to replace the replay simulator.
Vibration and acoustic channels	The strongest real-world predictive maintenance signals, absent from the AI4I dataset.
Online learning	Periodic retraining as real machine data accumulates, with drift monitoring.
Per-machine RUL calibration	Replace the population-mean nominal service life with a per-machine estimate learned from its own history.
Maintenance scheduling optimiser	Convert remaining-life estimates across a plant into an ordered, resource-aware work queue.
Multi-user operation	Authentication, per-technician alarm assignment and an audit trail.
Mobile notification	Push alerts by SMS or email, as described in the synopsis methodology.

References

Works reviewed in the literature survey

- [1] S. Rani S, R. Aburukba and K. El-Fakih, "Optimizing Predictive Maintenance in Industrial IoT Cloud Using Dragonfly Algorithm," IEEE Internet of Things Journal, 2025.
- [2] L. Cummins, A. Sommers et al., "Explainable Predictive Maintenance: A Survey of Current Methods, Challenges and Opportunities," 2024.
- [3] J. M. Montes-Sánchez, Y. Uwate et al., "Predictive Maintenance Edge AI Using RNN for Peristaltic Pumps," IEEE, 2025.
- [4] T. Akyaz and D. Engin, "Machine Learning-Based Predictive Maintenance System for Artificial Yarn Machines," IEEE, 2024.
- [5] S. D. Deshmukh and P. U. Dere, "AI Driven Approach for Predictive Maintenance in Industry 4.0," International Journal of Engineering Research & Technology (IJERT).
- [6] C. R. Patil, S. K. Jadhav et al., "Machine Learning-Based Predictive Maintenance of Industrial Machines," International Journal of Computer Trends and Technology, March 2023.
- [7] D. Guidotti, L. Pandolfo and L. Pulina, "A Systematic Literature Review of Supervised Machine Learning Techniques for Predictive Maintenance in Industry 4.0," 2025.
- [8] E. Zeydan, S. S. Arslan, Y. Turk, T. Hewa and M. Liyanage, "The Role of Mobile Communications for Industrial Automation: Architecture, Applications and Challenges," 2025.
- [9] K. Patel, A. Hariharan and R. Sucharitha, "Implementing Next-Gen Predictive Maintenance in Industrial Machineries: A Comparative Analysis of Small, Medium and Large Enterprises," 2023.
- [10] S. Gawde, S. Patil, S. Kumar and P. Kamat, "Explainable Predictive Maintenance of Rotating Machines Using LIME, SHAP, PDP and ICE," 2023.

Additional technical sources used in the implementation

- [11] S. Matzka, "Explainable Artificial Intelligence for Predictive Maintenance Applications," Third International Conference on Artificial Intelligence for Industries, 2020. (Source of the AI4I 2020 dataset.)
- [12] AI4I 2020 Predictive Maintenance Dataset, UCI Machine Learning Repository.
<https://archive.ics.uci.edu/dataset/601/>
- [13] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
- [14] A. Saabas, "Interpreting Random Forests," 2014. (Decision-path attribution, the method used for the explainability panel.)
- [15] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS, 2017. (TreeSHAP generalises the above.)
- [16] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [17] A. Saxena, K. Goebel, D. Simon and N. Eklund, "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation," IEEE PHM, 2008. (Run-to-failure methodology.)

Appendix A - How to Run the System

```
cd "AI BASED PREDICTIVE MAINTENCE FOR INDUSTRIAL MACHINES"

# start the system (API and interface on one port)
.\.venv\Scripts\python.exe -m backend.app

# then open http://127.0.0.1:8000/ and press 'Start Demo Plant'
```

Rebuilding the model from scratch (about 10 minutes):

```
.\.venv\Scripts\python.exe run_phase1.py

# or resume from a stage (1 download, 2 expand, 3 EDA, 4 train, 5 evaluate)
.\.venv\Scripts\python.exe run_phase1.py --from 4
```

Running the tests:

```
.\.venv\Scripts\python.exe -m pytest tests/ -q      # 57 engine + API tests
node tests/test_widgets.js                          # 29 widget assertions
node tests/test_boot.js                            # 17 boot assertions
```

Appendix B - Project Structure

```
src/          Phase 1 - data preparation, features, training, evaluation
  config.py   single source of truth: paths, schema, thresholds
  data/       download, physics-informed expansion, EDA
  features/   the shared feature builder
  models/     Random Forest training
  evaluation/ metrics, figures, early-warning and alarm-quality analysis

backend/      Phase 2 - inference engine and API
  app.py      FastAPI application
  core/       physics, health, alarms, database
  services/   predictor, explainer, simulator
  api/        routes and request schemas

frontend/     Phase 3 - SCADA interface (no build step)
  index.html, css/style.css, js/{api,widgets,app}.js, js/screens/*.js

tests/        57 Python tests + 46 JavaScript assertions
data/         raw and processed datasets, runtime SQLite database
models/       trained Random Forest bundle
artifacts/    13 analysis figures and metrics in JSON
docs/         the three project reports and their generators
```